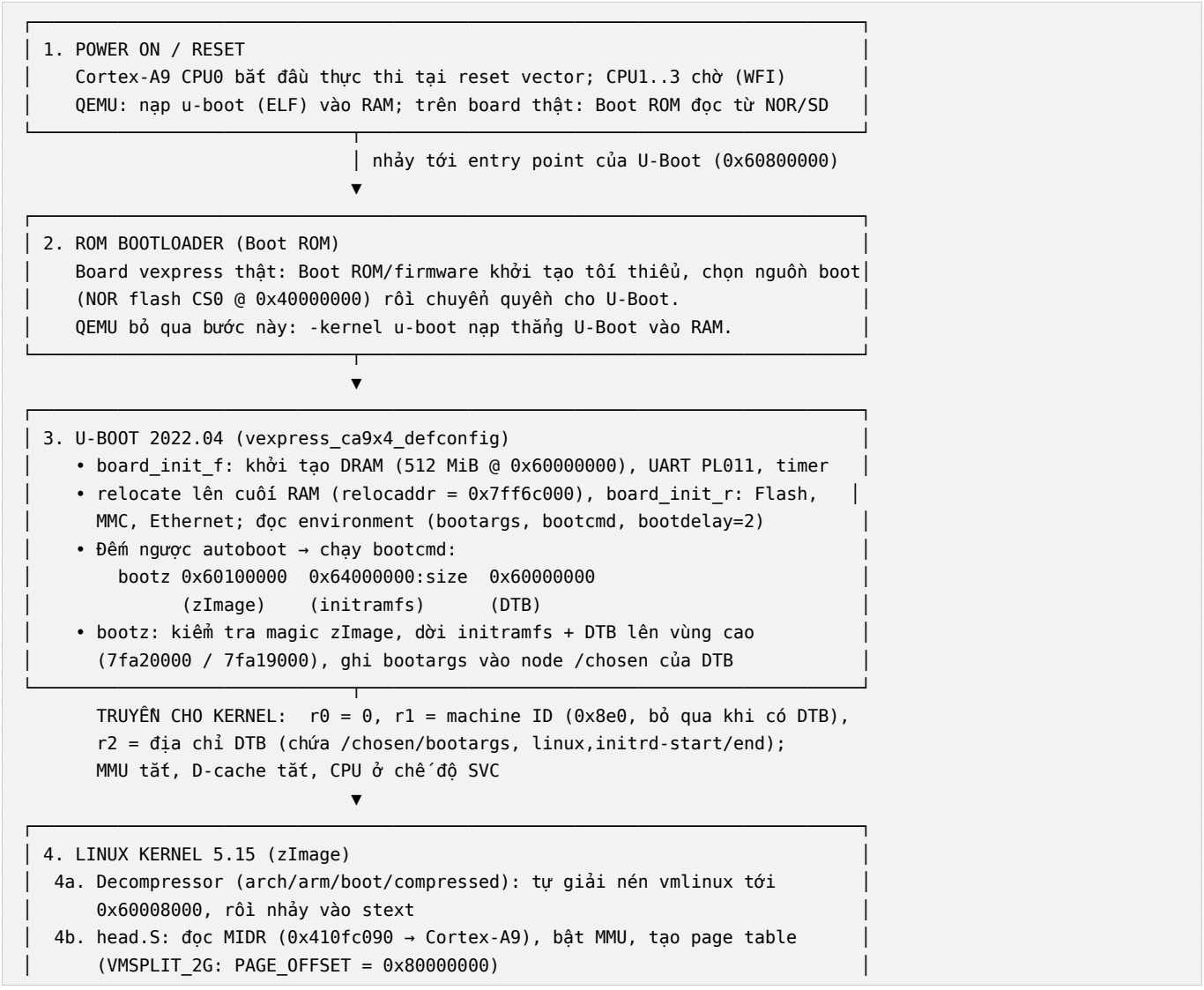
BÁO CÁO LAB-01: Cấu hình Kernel & Khởi động Hệ thống Linux Nhúng (ARMv7)

Embedded Linux Systems

Mục 1: Thông tin sinh viên

Mục	Nội dung
Họ và tên	Trần Đức Kiên
MSSV	SE183502
Ngày thực hiện	19/09/2026 (build), 12/10/2026 (thu thập log, hoàn thiện báo cáo)
Kiến trúc mục tiêu	ARMv7-A – Cortex-A9 (QEMU vexpress-a9)
Host	Ubuntu 26.04 LTS x86_64 (Linux 7.0.0-34-generic), i7-13620H 16 luồng
Toolchain	`arm-linux-gnueabihf-gcc` (Ubuntu 15.2.0-16ubuntu1) 15.2.0, GNU ld 2.46
QEMU	10.2.1 (`qemu-system-arm -M vexpress-a9`)
Mã nguồn	Linux 5.15 LTS, U-Boot 2022.04, BusyBox 1.35.0

Mục 2: Boot Sequence Diagram



```
|  4c. start_kernel(): setup_arch (parse DTB), "Kernel command line: ...",     |
|      mm_init, sched_init, GIC + timer (sp804/twd), console ttyAMA0           |
|  4d. rest_init() → kernel_init (PID 1) + kthreadd (PID 2)                    |
|      smp_init: bật CPU1 ("CPU1: thread -1, cpu 1 ...", "Brought up 2 CPUs") |
|      do_initcalls: driver PL011, MMCI, SMSC911x, CLCD, ...                   |
|  4e. populate_rootfs: giải nén initramfs.cpio.gz vào rootfs (tmpfs/ramfs)    |
|  4f. exec rdinit=/sbin/init (hoặc /init nếu không có rdinit=)                |
└──────────────────────────────┬──────────────────────────────────────────────┘
       TRUYỀN CHO USERSPACE: rootfs đã mount, /dev/console mở làm fd 0/1/2,
       argv/envp từ kernel command line
                               ▼
┌──────────────────────────────────────────────────────────────────────────────┐
| 5. INIT (BusyBox init, PID 1)                                                |
|    • (nếu qua /init: mount /proc, /sys rồi exec /sbin/init)                  |
|    • đọc /etc/inittab:                                                       |
|        ::sysinit:/etc/init.d/rcS   → mount proc, sysfs, tmpfs; hostname;     |
|                                      in "Boot completed! Welcome ..."        |
|        ::askfirst:-/bin/sh         → chờ Enter trên console                  |
└──────────────────────────────┬──────────────────────────────────────────────┘
                               ▼
┌──────────────────────────────────────────────────────────────────────────────┐
| 6. BUSYBOX SHELL  (-/bin/sh, login shell)                                    |
|    / # uname -a → Linux embedded-lab 5.15.0 ... armv7l GNU/Linux             |
└──────────────────────────────────────────────────────────────────────────────┘
```

Bước	Thành phần chạy	Công việc chính	Truyền cho bước tiếp theo
1. Power ON	Phần cứng CPU	Reset, CPU0 chạy reset vector, CPU phụ chờ	PC trỏ tới Boot ROM
2. ROM bootloader	Boot ROM / firmware (QEMU: mô phỏng bằng `-kernel u-boot`)	Khởi tạo tối thiểu, chọn thiết bị boot	Ảnh U-Boot trong RAM, nhảy vào entry
3. U-Boot	`u-boot` (ARMv7 ELF)	Khởi tạo DRAM/UART/Flash/MMC/Net, đọc env, chạy `bootcmd`, nạp zImage + initramfs + DTB	`r2` = DTB (có `bootargs`, `linux,initrd-start/end`), MMU off
4. Kernel	`zImage` → `vmlinux`	Giải nén, bật MMU, `start_kernel`, init driver, bật CPU phụ, giải nén initramfs	Rootfs trong RAM, exec `rdinit/init`
5. init	`/sbin/init` (BusyBox)	Đọc `inittab`, chạy `rcS` (mount proc/sys/tmp, hostname)	Spawn shell trên console
6. Shell	`/bin/sh` (BusyBox ash)	Tương tác người dùng	–

Mục 3: Log thực hành

Tất cả log đầy đủ nằm trong thư mục `logs/` của bài nộp.

3.1. Phần A – Cấu hình và build kernel

Cross-compile và toolchain (`logs/partA_defconfig.txt`):

```
$ export ARCH=arm CROSS_COMPILE=arm-linux-gnueabihf-
$ arm-linux-gnueabihf-gcc -Q --help=target | grep march
  -march=                               armv7-a+fp
```

`make vexpress_defconfig` (đề ghi `versatile_defconfig` ở mục 5.2, nhưng phần hướng dẫn A.2 dùng `vexpress_defconfig` – đúng cho Cortex-A9):

```
$ make vexpress_defconfig
  HOSTCC  scripts/kconfig/conf.o
```

```
  ...
  HOSTLD  scripts/kconfig/conf
#
# configuration written to .config
#
$ grep -E 'CONFIG_ARCH_VEXPRESS|CONFIG_ARM_LPAE|CONFIG_CPU_V7' .config
CONFIG_ARCH_VEXPRESS=y
CONFIG_ARCH_VEXPRESS_CORTEX_A5_A9_ERRATA=y
CONFIG_ARCH_VEXPRESS_DCSCB=y
CONFIG_ARCH_VEXPRESS_SPC=y
CONFIG_ARCH_VEXPRESS_TC2_PM=y
CONFIG_CPU_V7=y
# CONFIG_ARM_LPAE is not set
```

Các thay đổi trong menuconfig (đối chiếu `configs/kernel.config`):

Thay đổi	Đường dẫn menu	Symbol	Giá trị
1. Platform ARMv7	System Type → Platform selection → ARM Ltd. Versatile Express family	`CONFIG_ARCH_VEXPRESS`, `CONFIG_CPU_V7`	y
1. LPAE (quan sát)	System Type → Support for Large Physical Address Extension	`CONFIG_ARM_LPAE`	không bật (512 MB < 4 GB)
1. Memory split (quan sát)	Kernel Features → Memory split	`CONFIG_VMSPLIT_2G`, `CONFIG_PAGE_OFFSET=0x80000000`	2G/2G: user 0–2 GB, kernel 2–4 GB
2. initramfs (bắt buộc)	General setup → Initial RAM filesystem and RAM disk (initramfs/initrd) support	`CONFIG_BLK_DEV_INITRD`	y
3. Console	Device Drivers → Character devices → Serial drivers → ARM AMBA PL011 serial port support (+ console on PL011)	`CONFIG_SERIAL_AMBA_PL011`, `..._CONSOLE`	y
4. SMP (điểm cộng)	Kernel Features → Symmetric Multi-Processing	`CONFIG_SMP`, `CONFIG_NR_CPUS=8`	y

So sánh `.config` cuối cùng với file sinh ra bởi `vexpress_defconfig` cho thấy **không có khác biệt**: `vexpress_defconfig` đã bật sẵn cả 4 tùy chọn trên, nên trong menuconfig các bước 1–4 chỉ là xác nhận/quan sát rồi Save.

Log cuối build (`make -j16 zImage dtbs modules`, `logs/partA_build.txt`):

```
  LD      vmlinux
  SORTTAB vmlinux
  SYSMAP  System.map
  OBJCOPY arch/arm/boot/Image
  Kernel: arch/arm/boot/Image is ready
  AS      arch/arm/boot/compressed/piggy.o
  LD      arch/arm/boot/compressed/vmlinux
  OBJCOPY arch/arm/boot/zImage
  Kernel: arch/arm/boot/zImage is ready

real    1m57.162s
user    24m58.122s
sys     2m14.874s
```

Kiểm tra kết quả (`logs/partA_verify.txt`):

```
$ ls -lh arch/arm/boot/zImage arch/arm/boot/dts/vexpress-v2p-ca9.dtb
-rw-rw-r-- 1 kien kien  14K Sep 19 08:25 arch/arm/boot/dts/vexpress-v2p-ca9.dtb
-rwxrwxr-x 1 kien kien 4.9M Oct 12 16:49 arch/arm/boot/zImage

$ file arch/arm/boot/zImage
```

```
arch/arm/boot/zImage: Linux kernel ARM boot executable zImage (kernel >=v4.15) (little-endian)

$ arm-linux-gnueabihf-readelf -A arch/arm/boot/compressed/vmlinux | grep -i 'arch\|cpu'
  Tag_CPU_name: "7-A"
  Tag_CPU_arch: v7
  Tag_CPU_arch_profile: Application
```

`zImage` 4,9 MB, đúng kiến trúc ARMv7-A. `make modules` không sinh file `.ko` nào vì `vexpress_defconfig` biên dịch toàn bộ driver vào kernel (`=y`), nhưng vẫn tạo `Module.symvers` – cần cho việc build driver out-of-tree ở LAB-02.

> **Ghi chú kỹ thuật – `KCFLAGS` trong mục A.1 làm build thất bại.** Khi đặt `KCFLAGS="-march=armv7-a -mfpu=vfpv3 -mfloat-abi=hard"` như đề, gcc 15 báo lỗi (`logs/partA_build_KCFLAGS_loi.txt`): `arch/arm/vfp/vfpmodule.c: Error: D register out of range for selected VFP version -- vldr.64 d16,...` và `arch/arm/vdso/Makefile:54: vdso.so.dbg] Error 1`. Nguyên nhân: kernel tự chọn cờ FPU riêng cho từng phần (`-mfloat-abi=soft -msoft-float` cho toàn kernel, `-mfpu=vfp` cho mã VFP), `KCFLAGS` được nối vào sau và ghi đè các cờ đó. Kernel không dùng FPU trong kernel space nên **không cần** `KCFLAGS` – chỉ `ARCH` và `CROSS_COMPILE` là đủ. Kernel được build thành công khi bỏ `KCFLAGS`.

3.2. Phần B – U-Boot

Build – `make vexpress_ca9x4_defconfig && make -j16` → `output/u-boot`: `ELF 32-bit LSB executable, ARM, EABI5, Tag_CPU_arch: v7`.

Khởi động U-Boot trên QEMU (`logs/partB_uboot.txt`):

```
U-Boot 2022.04 (Sep 19 2026 - 09:23:24 +0700)

DRAM:  512 MiB
WARNING: Caches not enabled
Core:  12 devices, 7 uclasses, devicetree: embed
Flash: 64 MiB
MMC:   mmci@5000: 0
Loading Environment from Flash... *** Warning - bad CRC, using default environment
In:    serial
Out:   serial
Err:   serial
Net:   eth0: ethernet@3,02000000
Hit any key to stop autoboot:  2  0
=> version
U-Boot 2022.04 (Sep 19 2026 - 09:23:24 +0700)
arm-linux-gnueabihf-gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
GNU ld (GNU Binutils for Ubuntu) 2.46
=> bdinfo
boot_params = 0x60002000
DRAM bank   = 0x00000000
-> start    = 0x60000000
-> size     = 0x20000000
flashstart  = 0x40000000
flashsize   = 0x04000000
baudrate    = 38400 bps
relocaddr   = 0x7ff6c000
reloc off   = 0x1f76c000
ethaddr     = 52:54:00:12:34:56
fdt_blob    = 0x7ffe0430
arch_number = 0x000008e0
...
```

`bdinfo`: **DRAM 512 MiB bắt đầu tại `0x60000000`**, NOR flash 64 MiB tại `0x40000000`; U-Boot đã tự relocate lên cuối RAM (`0x7ff6c000`).

`printenv` (trích các biến liên quan, đầy đủ trong log):

```
=> printenv
arch=arm
baudrate=38400
board=vexpress
bootargs=console=tty0 console=ttyAMA0,38400n8
bootcmd=run distro_bootcmd; run bootflash
bootdelay=2
cpu=armv7
fdt_addr_r=0x60000000
fdtfile=vexpress-v2p-ca9.dtb
kernel_addr_r=0x60100000
loadaddr=0x90000000
...
Environment size: 4238/262140 bytes
```

Các biến mặc định `kernel_addr_r=0x60100000` và `fdt_addr_r=0x60000000` **trùng với địa chỉ trong `bootcmd` của đề**.

`setenv bootargs` / `bootcmd` / `saveenv`:

```
=> setenv bootargs 'console=ttyAMA0,115200 root=/dev/ram0 rw init=/sbin/init mem=512M'
=> printenv bootargs
bootargs=console=ttyAMA0,115200 root=/dev/ram0 rw init=/sbin/init mem=512M
=> setenv bootcmd 'bootz 0x60100000 0x64000000 0x60000000'
=> printenv bootcmd
bootcmd=bootz 0x60100000 0x64000000 0x60000000
=> saveenv
Saving Environment to Flash... Error: start and/or end address not on sector boundary
Error: start and/or end address not on sector boundary
Failed (1)
```

`saveenv` thất bại như đề đã lưu ý: vùng environment cấu hình cho flash thật không khớp ranh giới sector của flash giả lập trong QEMU, nên mỗi lần khởi động lại U-Boot dùng default environment (`bad CRC`).

Kernel panic khi chưa có rootfs (`logs/partB_kernel_panic.txt`):

```
$ qemu-system-arm -M vexpress-a9 -cpu cortex-a9 -m 512M -nographic -kernel output/zImage \
    -dtb output/vexpress-v2p-ca9.dtb -append 'console=ttyAMA0,115200 root=/dev/ram0 rw mem=512M'
Booting Linux on physical CPU 0x0
CPU: ARMv7 Processor [410fc090] revision 0 (ARMv7), cr=10c5387d
Kernel command line: console=ttyAMA0,115200 root=/dev/ram0 rw mem=512M
...
VFS: Cannot open root device "ram0" or unknown-block(0,0): error -6
Please append a correct "root=" boot option; here are the available partitions:
1f00          131072 mtdblock0
1f01           32768 mtdblock1
Kernel panic - not syncing: VFS: Unable to mount root fs on unknown-block(0,0)
CPU: 0 PID: 1 Comm: swapper/0 Not tainted 5.15.0 #1
Hardware name: ARM-Versatile Express
[<80863f64>] (panic) from [<80b01714>] (mount_block_root+0x1bc/0x264)
[<80b01714>] (mount_block_root) from [<80b01b98>] (prepare_namespace+0x148/0x184)
[<80b01b98>] (prepare_namespace) from [<8086c6b8>] (kernel_init+0x14/0x13c)
---[ end Kernel panic - not syncing: VFS: Unable to mount root fs on unknown-block(0,0) ]---
```

Đề dự đoán `unknown-block(1,0)`, kết quả thực tế là `unknown-block(0,0)`: kernel **không bật `CONFIG_BLK_DEV_RAM`** nên không có driver RAM disk (major 1) – tên `ram0` không phân giải được thành thiết bị nào → `(0,0)`, `error -6` (`ENXIO`). Kernel chỉ thấy 2 phân vùng flash `mtdblock0/1`. Backtrace cho thấy đường đi `kernel_init → prepare_namespace → mount_block_root → panic`: không có initramfs nên kernel cố mount root fs từ thiết bị khối theo `root=`.

Boot đầy đủ U-Boot → `bootz` → kernel → shell (`logs/partB_uboot_bootz.txt`) – chạy đúng `bootcmd` với 3 địa chỉ của đề, QEMU đặt sẵn file vào RAM bằng `-device loader` (thay cho việc U-Boot `load` từ SD/TFTP trên board thật):

```
=> setenv bootargs 'console=ttyAMA0,115200 root=/dev/ram0 rw init=/sbin/init mem=512M'
=> setenv bootcmd 'cp.b 0x68000000 0x60000000 0x3701; bootz 0x60100000 0x64000000:0x10783d 0x60000000'
=> run bootcmd
Kernel image @ 0x60100000 [ 0x000000 - 0x4e0b60 ]
## Flattened Device Tree blob at 60000000
   Booting using the fdt blob at 0x60000000
   Loading Ramdisk to 7fa20000, end 7fb2783d ... OK
   Loading Device Tree to 7fa19000, end 7fa1f700 ... OK

Starting kernel ...

Booting Linux on physical CPU 0x0
Linux version 5.15.0 (kien@kien-Nitro-ANV15-52) (arm-linux-gnueabihf-gcc ... 15.2.0 ...) #1 SMP ...
Kernel command line: console=ttyAMA0,115200 root=/dev/ram0 rw init=/sbin/init mem=512M
...
Run /init as init process
Boot completed! Welcome to Embedded Linux Lab 1
/ # uname -a
Linux embedded-lab 5.15.0 #1 SMP Sat Sep 19 07:59:31 +07 2026 armv7l GNU/Linux
/ # cat /proc/cmdline
console=ttyAMA0,115200 root=/dev/ram0 rw init=/sbin/init mem=512M
```

Kernel nhận đúng `bootargs` do U-Boot ghi vào DTB. Vì có initramfs, kernel chạy `/init` trong initramfs (bỏ qua `root=/dev/ram0`), `/init` mount proc/sys rồi `exec /sbin/init`.

> **Ghi chú kỹ thuật – 3 điều chỉnh cần thiết để `bootz` chạy được trên QEMU:** 1. **Kích thước initramfs:** với initramfs dạng raw (không có header uImage), `bootz` cần dạng `addr:size` (`0x64000000:0x10783d`); chỉ ghi `0x64000000` như đề thì U-Boot không biết initramfs dài bao nhiêu. 2. **DTB bị ghi đè:** nạp sẵn DTB tại `0x60000000` trước khi U-Boot chạy thì bị lỗi `fdt_find_or_add_subnode: chosen: FDT_ERR_BADSTRUCTURE`. So sánh bằng `cmp.b` cho thấy dữ liệu tại `0x60000000` khác bản gốc từ byte `0x688` – U-Boot dùng vùng đầu RAM (`boot_params = 0x60002000`, stack/dữ liệu sớm) trong lúc khởi động. Trên board thật U-Boot nạp DTB **sau khi** khởi động xong nên không gặp lỗi; trong QEMU, DTB được đặt tạm ở `0x68000000` rồi `cp.b` sang `0x60000000` – địa chỉ `bootz` vẫn đúng như đề. 3. **Không dùng `-smp 2` khi `-kernel` là U-Boot:** QEMU cho mọi CPU cùng chạy U-Boot (không có boot stub đưa CPU phụ vào WFI như khi `-kernel` là zImage) → không có output. Bản U-Boot này cũng không có lệnh `boot` (dùng `run bootcmd`).

3.3. Phần C – Root filesystem và initramfs

BusyBox (`configs/busybox.config`): `CONFIG_STATIC=y`, `CONFIG_CROSS_COMPILER_PREFIX="arm-linux-gnueabihf-"`, `CONFIG_EXTRA_CFLAGS="-march=armv7-a -mfpu=neon-vfpv3 -mfloat-abi=hard"`, `# CONFIG_TC is not set`.

```
$ file rootfs/initramfs/bin/busybox
ELF 32-bit LSB executable, ARM, EABI5 version 1 (GNU/Linux), statically linked, ..., stripped
$ arm-linux-gnueabihf-readelf -A rootfs/initramfs/bin/busybox | grep -E 'Tag_CPU|Tag_ABI'
  Tag_CPU_arch: v7
  Tag_ABI_VFP_args: VFP registers        ← hard-float ABI
```

`ls initramfs/` (`logs/partC_rootfs.txt`):

```
$ ls -la rootfs/initramfs/
drwxrwxr-x  2 kien kien 4096 bin
drwxrwxr-x  2 kien kien 4096 dev
drwxrwxr-x  3 kien kien 4096 etc
-rwxrwxr-x  1 kien kien   76 init
drwxrwxr-x  3 kien kien 4096 lib
drwxrwxr-x  2 kien kien 4096 proc
drwxrwxr-x  2 kien kien 4096 sbin
drwxrwxr-x  2 kien kien 4096 sys
```

```
drwxrwxr-x  2 kien kien 4096 tmp
drwxrwxr-x  4 kien kien 4096 usr
$ ls -l rootfs/initramfs/dev
crw-------  1 root root 5, 1 console
crw-rw-rw-  1 root root 1, 3 null
cr--r--r--  1 root root 1, 8 random
crw-rw-rw-  1 root root 1, 5 zero
$ ls -lh output/initramfs.cpio.gz
-rw-rw-r-- 1 kien kien 1.1M output/initramfs.cpio.gz
```

File cấu hình: `/init` (mount proc, sysfs, `exec /sbin/init`), `/etc/inittab` (`sysinit` → `rcS`, `askfirst` → `-/bin/sh`), `/etc/init.d/rcS` (mount proc/sys/tmpfs, in thông báo, `hostname embedded-lab`), `/etc/passwd`, `/etc/hostname`.

Boot thành công (`logs/partC_boot_smp2.txt`):

```
$ qemu-system-arm -M vexpress-a9 -cpu cortex-a9 -m 512M -smp 2 -nographic \
    -kernel ~/embedded_lab1/output/zImage -dtb ~/embedded_lab1/output/vexpress-v2p-ca9.dtb \
    -initrd ~/embedded_lab1/output/initramfs.cpio.gz \
    -append 'console=ttyAMA0,115200 rdinit=/sbin/init mem=512M'
Booting Linux on physical CPU 0x0
CPU: ARMv7 Processor [410fc090] revision 0 (ARMv7), cr=10c5387d
CPU: PIPT / VIPT nonaliasing data cache, VIPT nonaliasing instruction cache
...
smp: Bringing up secondary CPUs ...
CPU1: thread -1, cpu 1, socket 0, mpidr 80000001
smp: Brought up 1 node, 2 CPUs
SMP: Total of 2 processors activated (1834.59 BogoMIPS).
...
Run /sbin/init as init process
Boot completed! Welcome to Embedded Linux Lab 1

Please press Enter to activate this console.
```

Các lệnh kiểm tra trong BusyBox shell:

```
/ # uname -a
Linux embedded-lab 5.15.0 #1 SMP Sat Sep 19 07:59:31 +07 2026 armv7l GNU/Linux
/ # cat /proc/cpuinfo | grep -E 'processor|model|Features'
processor       : 0
model name      : ARMv7 Processor rev 0 (v7l)
Features        : half thumb fastmult vfp edsp neon vfpv3 tls vfpd32
processor       : 1
model name      : ARMv7 Processor rev 0 (v7l)
Features        : half thumb fastmult vfp edsp neon vfpv3 tls vfpd32
/ # cat /proc/cpuinfo        (trích)
CPU implementer : 0x41
CPU architecture: 7
CPU variant     : 0x0
CPU part        : 0xc09
CPU revision    : 0
Hardware        : ARM-Versatile Express
/ # cat /proc/meminfo | head -5
MemTotal:         508288 kB
MemFree:          493112 kB
MemAvailable:     490348 kB
Buffers:               0 kB
Cached:             1568 kB
/ # ls /dev
console  null     random   zero
/ # mount
rootfs on / type rootfs (rw,size=244912k,nr_inodes=61228)
none on /proc type proc (rw,relatime)
none on /sys type sysfs (rw,relatime)
none on /tmp type tmpfs (rw,relatime)
```

```
/ # ps
PID   USER     TIME  COMMAND
    1 root      0:02 init
    2 root      0:00 [kthreadd]
    9 root      0:00 [ksoftirqd/0]
   11 root      0:00 [migration/0]
   14 root      0:00 [migration/1]
   15 root      0:00 [ksoftirqd/1]
   ...
   59 root      0:00 -/bin/sh
   68 root      0:00 ps
/ # cat /sys/devices/system/cpu/possible
0-3
/ # cat /sys/devices/system/cpu/online
0-1
```

Nhận xét kết quả:

- `armv7l`, `CPU part 0xc09` (Cortex-A9), Features có `vfpv3 neon vfpd32` – xác nhận đúng ARMv7-A có FPU/NEON. `MemTotal` 508 288 kB ≈ 512 MB (phần thiếu là vùng kernel code/reserved).
- `ls /dev` chỉ có 4 node tạo bằng `mknod` vì rootfs không mount devtmpfs; `mount` có đúng proc, sysfs, tmpfs do `rcS` mount.
- `possible = 0-3` chứ không phải `0-1` như đề dự đoán: `possible` lấy từ số CPU khai báo trong DTB `vexpress-v2p-ca9.dtb` (4 node CPU), còn `online = 0-1` mới phản ánh `-smp 2`. Boot log tương ứng có `CPU2: failed to boot: -38` và `CPU3: failed to boot: -38` (`-ENOSYS`: QEMU không có CPU 2, 3 để đánh thức).
- `ps` có các kernel thread theo từng CPU (`migration/N`, `ksoftirqd/N`, `cpuhp/N`) cho N = 0–3 do `nr_cpu_ids = 4`.

> **Ghi chú kỹ thuật – `/init` đã được sửa theo đề.** Bản `/init` cũ (đến 19/09) mount thêm devtmpfs rồi `exec /bin/sh` – bỏ qua BusyBox init, `inittab` và `rcS` nếu kernel chạy `/init`. Lệnh boot của mục C.5 dùng `rdinit=/sbin/init` nên không bị ảnh hưởng, nhưng khi boot qua U-Boot với `init=/sbin/init` (không có `rdinit=`), kernel chạy `/init`. `/init` hiện tại giống đề: mount proc, sysfs rồi `exec /sbin/init`; initramfs được đóng gói lại.

Mục 4: Phân tích và trả lời câu hỏi

Câu 1. `arm-linux-gnueabi` và `arm-linux-gnueabihf`

	arm-linux-gnueabi	**arm-linux-gnueabihf**
ABI gọi hàm với số thực	*soft-float ABI*: tham số/giá trị trả về `float`/`double` truyền qua thanh ghi **nguyên** `r0`–`r3` và stack	*hard-float ABI* (AAPCS-VFP): truyền qua thanh ghi **VFP** `s0`–`s15`/`d0`–`d7`
Phép toán số thực	mặc định gọi thư viện mô phỏng (`__aeabi_dadd`...), hoặc `softfp` (dùng lệnh VFP bên trong hàm nhưng vẫn truyền qua `r0`–`r3`)	lệnh VFP/NEON phần cứng trực tiếp
Kiến trúc mặc định của Debian/Ubuntu	ARMv5TE trở lên (armel)	ARMv7-A + VFPv3-D16 + Thumb-2 (armhf)
Dynamic linker	`/lib/ld-linux.so.3`	`/lib/ld-linux-armhf.so.3`
Thuộc tính ELF	`Tag_ABI_VFP_args` không có	`Tag_ABI_VFP_args: VFP registers` (BusyBox của bài: có)

Vì sao Cortex-A9 dùng `gnueabihf`: Cortex-A9 có FPU VFPv3 và NEON (thấy trong `Features: vfp neon vfpv3 vfpd32`). Hard-float dùng trực tiếp thanh ghi VFP để truyền tham số nên tránh được việc chép qua lại giữa thanh ghi nguyên và VFP mỗi lần gọi hàm, và không phải mô phỏng float bằng phần mềm (chậm hàng chục lần). Toàn bộ userspace của Ubuntu armhf cũng theo ABI này.

Build kernel bằng `gnueabi` rồi chạy trên Cortex-A9: kernel **vẫn chạy bình thường**. `arch/arm/Makefile` của kernel luôn ép `CFLAGS_ABI := -mabi=aapcs-linux -mfloat-abi=soft` và `-msoft-float` (dòng 103, 128) – kernel không dùng FPU trong kernel space (FPU chỉ được bật lười cho userspace và lưu/khôi phục khi context switch), nên kernel build từ 2 toolchain cho cùng mã máy về mặt ABI. Chính lỗi `KCFLAGS` ở Phần A cho thấy điều này: ép `-mfloat-abi=hard` vào kernel còn làm build hỏng. Vấn đề chỉ xuất hiện ở **userspace**: binary `gnueabi` liên kết động sẽ tìm `/lib/ld-linux.so.3` và libc soft-float – không tồn tại trong rootfs armhf → `not found`; trộn thư viện hai ABI thì sai giá trị số thực khi gọi hàm (tham số nằm ở `r0–r3` nhưng bên nhận đọc ở `d0`). Với BusyBox static thì chạy được nhưng mọi phép tính số thực bị chậm do mô phỏng.

Câu 2. Memory map vexpress-a9 bắt đầu từ `0x60000000`

Memory map do **thiết kế phần cứng (SoC/bus)** quyết định, không phải do phần mềm. Trên Versatile Express (bản đồ "legacy" của daughterboard CoreTile A9):

Vùng	Địa chỉ
NOR flash 0/1 (CS0/CS1), SRAM, thiết bị motherboard (UART, MMC, Ethernet...)	`0x40000000 – 0x5FFFFFFF`
Ngoại vi của daughterboard (CLCD, DMC, SCU/GIC private)	`0x10000000 – 0x1FFFFFFF`, `0x1E000000`
DRAM (tối đa 1 GB, QEMU 512 MB)	**`0x60000000 – 0x7FFFFFFF`** (U-Boot: `memory[0] [0x60000000-0x80000003]`)

Vùng địa chỉ thấp dành cho ngoại vi và flash (để Boot ROM/flash có thể nằm ở vector reset và chip select đơn giản), nên RAM được đặt từ `0x60000000`. Versatile PB (ARM926) là board cũ hơn với DRAM ở `0x00000000`.

Ảnh hưởng tới `bootcmd`: mọi địa chỉ nạp phải nằm trong `0x60000000–0x7FFFFFFF` và không chồng lấn:

- DTB `0x60000000` (14 KB, đến `0x60003701`), zImage `0x60100000` (4,9 MB, đến `0x605E0B60`), initramfs `0x64000000` (1 MB). Khi kernel tự giải nén, nó ghi `vmlinux` tới `0x60008000` (= `PHYS_OFFSET + TEXT_OFFSET 0x8000`); zImage được đặt cách 1 MB phía trên để decompressor tự dời mình khi chồng lấn, và DTB nằm dưới `0x60008000` nên không bị đè.
- Trong log `bootz`, U-Boot còn tự dời initramfs và DTB lên cuối RAM (`7fa20000`, `7fa19000`) để tránh vùng kernel giải nén vào.
- Không thể dùng các địa chỉ của Versatile PB (ví dụ `bootz 0x00010000 ...`): ở vexpress `0x0` là vùng ánh xạ flash/không có RAM.

Nếu đặt sai địa chỉ load: (1) địa chỉ ngoài DRAM → lệnh nạp/`cp` báo lỗi hoặc CPU bị *data abort* và U-Boot reset; (2) `bootz` trỏ vào vùng không chứa zImage → `Bad Linux ARM zImage magic!`; (3) các vùng chồng lấn nhau hoặc chồng lên vùng U-Boot đang dùng → dữ liệu bị đè: thực tế trong bài, DTB nạp sẵn ở `0x60000000` bị U-Boot ghi đè và `bootz` báo `FDT_ERR_BADSTRUCTURE`, `FDT creation failed! resetting ...`; nếu initramfs bị đè thì kernel báo `Initramfs unpacking failed` rồi panic như Phần B.

Câu 3. MIDR `0x410fc090`

MIDR (Main ID Register, CP15 `c0`) được kernel đọc ở `head.S` để chọn `proc_info` phù hợp:

```
0x410fc090 = 0100 0001 | 0000 | 1111 | 1100 0000 1001 | 0000
             [31:24]    [23:20] [19:16]  [15:4]          [3:0]
```

Trường	Bit	Giá trị	Ý nghĩa
Implementer	[31:24]	`0x41`	ASCII `'A'` = **ARM Limited**
Variant	[23:20]	`0x0`	major revision → **r0**
Architecture	[19:16	0xF	"định nghĩa theo CPUID scheme" – dùng cho ARMv7 trở lên (kiến trúc thật đọc từ

	]		thanh ghi ID_MMFR*)
Part number	[15:4]	0xC09	**Cortex-A9** (0xC08 = Cortex-A8, 0xC0F = Cortex-A15, 0xC07 = Cortex-A7)
Revision	[3:0]	0x0	minor revision → **p0**

→ CPU là **ARM Cortex-A9 r0p0**. Kết quả khớp với `/proc/cpuinfo` trong QEMU: `CPU implementer 0x41`, `CPU architecture: 7`, `CPU variant 0x0`, `CPU part 0xc09`, `CPU revision 0`. (Ví dụ output U-Boot trong đề ghi `Cortex-A9 r3p0` – board thật; QEMU mô phỏng r0p0.) Phần `cr=10c5387d` trong cùng dòng log là giá trị thanh ghi SCTLR sau khi bật MMU (bit 0 M = MMU on, bit 2 C = D-cache, bit 12 I = I-cache, bit 13 V = vector cao `0xFFFF0000`).

Câu 4. Static linking cho BusyBox trong initramfs

Vì sao static: initramfs tối giản chỉ có BusyBox – không có `/lib/ld-linux-armhf.so.3` hay libc. Binary liên kết động khi `exec` sẽ được kernel nạp kèm dynamic linker ghi trong `PT_INTERP`; nếu file đó không tồn tại thì `execve` trả `ENOENT` và kernel báo `Failed to execute /sbin/init (error -2)` → panic `No working init found`, dù file `/sbin/init` có mặt. Static linking đưa mọi phần cần thiết của libc vào chính binary (1,6 MB) nên rootfs không phụ thuộc gì, dễ đóng gói và không lo lệch phiên bản glibc giữa toolchain và rootfs.

Nếu build dynamic cho ARMv7, phải chép từ sysroot của toolchain `/usr/arm-linux-gnueabihf/lib/` vào `rootfs/initramfs/lib/`:

File	Vai trò
`ld-linux-armhf.so.3`	dynamic linker/loader (đường dẫn cố định trong `PT_INTERP` của binary armhf)
`libc.so.6`	thư viện C chuẩn (glibc)
`libm.so.6`	thư viện toán – BusyBox link `-lm` (`Makefile.flags`: `LDLIBS += m`)
`libresolv.so.2`	phân giải DNS cho các applet mạng (`LDLIBS += resolv`)
(tùy cấu hình) `libcrypt.so.1`	`crypt()` cho `passwd`, `login` (`LDLIBS += crypt`)

Danh sách chính xác lấy bằng `arm-linux-gnueabihf-readelf -d busybox | grep NEEDED`. Cách làm này đã được áp dụng thực tế ở LAB-03: rootfs được thêm `ld-linux-armhf.so.3`, `libc.so.6`, `libm.so.6`, `libgcc_s.so.1`, `libstdc++.so.6` từ `/usr/arm-linux-gnueabihf/lib` để chạy `gdbserver`/`strace` liên kết động.

Câu 5. SMP: `-smp 1` và `-smp 2`

Kernel SMP cần khởi tạo thêm so với uniprocessor:

- Cấu trúc dữ liệu theo từng CPU (`percpu` areas, runqueue cho mỗi CPU, `nr_cpu_ids`, RCU theo số CPU).
- **Đánh thức CPU phụ**: CPU0 ghi địa chỉ vật lý của `versatile_secondary_startup` vào thanh ghi `SYS_FLAGS` của motherboard (`vexpress_flags_set()` trong `arch/arm/mach-vexpress/platsmp.c`) rồi gửi IPI; CPU phụ thoát WFI, bật MMU, chạy `secondary_start_kernel`.
- Bộ điều khiển ngắt GIC phân phối ngắt và IPI giữa các core; SCU (Snoop Control Unit) giữ cache L1 các core nhất quán; mỗi core có local timer (TWD) riêng.
- Kernel thread theo từng CPU: `migration/N`, `ksoftirqd/N`, `cpuhp/N`, `kworker/N:*`.
- Spinlock thực sự (`ldrex`/`strex` + barrier) thay vì chỉ tắt preemption, cộng thêm chi phí đồng bộ cache.

Khác biệt trong boot log (`logs/partC_boot_smp1.txt` và `partC_boot_smp2.txt`, cùng zImage SMP):

`-smp 1`	`-smp 2`	Ý nghĩa
`smp: Bringing up secondary CPUs ...` rồi ngay `smp: Brought up 1 node, 1 CPU`	`CPU1: thread -1, cpu 1, socket 0, mpidr 80000001` / `CPU1: Spectre v2: using BPIALL workaround` / `smp: Brought up 1 node, 2 CPUs`	CPU1 (MPIDR `...01`) được đánh thức và chạy `secondary_start_kernel`, áp workaround Spectre riêng cho core đó
`SMP: Total of 1`	`SMP: Total of 2 processors`	Tổng BogoMIPS ≈ 2 × mỗi core (904 + 930)

`processors activated (927.33 BogoMIPS).`	`activated (1834.59 BogoMIPS).`	
–	`CPU2: failed to boot: -38`, `CPU3: failed to boot: -38`	DTB khai báo 4 CPU (`possible 0-3`), QEMU chỉ có 2 → CPU 2, 3 không phản hồi (`-ENOSYS`)
`GIC CPU mask not found - kernel will fail to boot.` (2 lần)	không có	Với 1 CPU, driver GIC đọc thanh ghi target của GIC mô phỏng trả về 0 (không có CPU interface nào khác) nên cảnh báo; kernel vẫn boot bình thường
`/proc/cpuinfo`: 1 processor; `online = 0`	2 processor; `online = 0-1`	Số CPU online đúng với `-smp`

Ảnh hưởng đến kích thước image và thời gian boot (yêu cầu ghi chú của Thay đổi 4): bật `CONFIG_SMP` thêm mã khởi động CPU phụ, IPI, cấu trúc per-CPU và spinlock thật – kernel lớn hơn vài chục đến ~100 KB và mỗi thao tác khóa tốn thêm barrier. Thời gian boot tăng một chút cho bước đánh thức CPU phụ (log: ngay sau `Bringing up secondary CPUs` là `Brought up`), nhưng sau đó initcall và userspace có thể chạy song song trên 2 core. Trong bài, kernel `vexpress_defconfig` đã bật sẵn SMP (`CONFIG_SMP=y`, `NR_CPUS=8`), cùng một zImage chạy được cả `-smp 1` lẫn `-smp 2`.

Mục 5: Nhận xét

- Chuỗi khởi động được kiểm chứng đầy đủ ở cả hai cách: QEMU nạp thẳng zImage (`-kernel`, Phần C) và đúng chuỗi U-Boot → `bootz` → kernel → init → shell (Phần B), với `bootargs` U-Boot truyền vào xuất hiện nguyên vẹn trong `/proc/cmdline`.
- Đọc log kernel cho biết rất nhiều thông tin phần cứng: MIDR → loại CPU, `bdinfo`/`memory[0]` → memory map, `SMP: Total of N processors` → số core, `unknown-block(0,0)` → driver RAM disk không được bật.
- Địa chỉ nạp trong `bootcmd` không chỉ phải nằm trong RAM mà còn phải tránh vùng U-Boot đang dùng và vùng kernel giải nén – lỗi DTB bị ghi đè trong bài là ví dụ cụ thể.
- Kernel ARM luôn được build soft-float bất kể toolchain; lựa chọn hard-float/soft-float chủ yếu quyết định ABI của userspace (BusyBox, thư viện).

Phụ lục: Danh sách file nộp

```
SE183502_TranDucKien_Lab01/
├── output/
│   ├── zImage                      # Kernel image ARMv7 (Linux 5.15)
│   ├── vexpress-v2p-ca9.dtb        # Device Tree Blob Cortex-A9
│   ├── initramfs.cpio.gz           # Root filesystem (đã cập nhật /init theo đề)
│   └── u-boot                      # U-Boot 2022.04 vexpress_ca9x4
├── configs/
│   ├── kernel.config               # .config của kernel
│   └── busybox.config              # .config của BusyBox
├── rootfs/initramfs/               # rootfs trước khi nén
├── logs/
│   ├── partA_defconfig.txt, partA_build.txt, partA_verify.txt
│   ├── partA_build_KCFLAGS_loi.txt # log lỗi khi dùng KCFLAGS của đề`
│   ├── partB_uboot.txt             # version, bdinfo, printenv, setenv, saveenv
│   ├── partB_kernel_panic.txt      # boot không có initramfs
│   ├── partB_uboot_bootz.txt       # U-Boot bootz → kernel → shell
│   ├── partC_rootfs.txt            # ls initramfs, file/readelf busybox, cpio -t
│   └── partC_boot_smp2.txt, partC_boot_smp1.txt
└── bao_cao/
    └── SE183502_Lab01_BaoCao.pdf
```